Name : AKASH SAMPATH

Module : All Weeks : 0 , 1 , 2 , 4 , 5 , 6

Topic : All Commands and Explanation

Project name : Online Examination System

Tools and Usage :

- **Frontend:** React
- **Backend:** Python + FastAPI
- **Database:** PostgreSQL
- **Authentication:** Keycloak
- **Containers:** Docker
- **Orchestration:** Docker Compose
- **Testing:** Swagger and Postman

Procedure : (Starting) :

Open visual studio code and click terminal and new terminal :

(**Go to project root folder)** :

```
cd "C:\Users\akash\OneDrive\Desktop\Project_Intern_Final_Wk_All\Exam\Exam"
```

(**Check current folder**) :

```
pwd
```

Week 0 : Full commands with meaning : (Project setup and Git basics) :

Goal : Keep your project organised and save your work in Git/GitHub.

No.	Command	Simple meaning
1	`pwd`	Show the current folder.
2	`dir`	Show files and folders.
3	`git --version`	Check whether Git is installed.
4	`git init`	Start Git tracking in this project. Run once if it is not already a Git repository.
5	`git status`	Show which files have changed.
6	`git add .`	Prepare all changed files for saving.
7	`git commit -m "Initial Online Examination System"`	Save a snapshot of your project in Git.
8	`git branch`	Show your current branch.
9	`git log --oneline`	Show saved commits in a short format.
10	`git tag v1.0`	Put a version label on the current commit.
11	`git tag`	Show your version tags.

Connection project to GitHub : (Only if you have created an empty GitHub repository):

```
git remote add origin YOUR_GITHUB_REPOSITORY_URL
```

Command	Simple meaning
`git remote -v`	Check the GitHub repository connection.
`git branch -M main`	Rename the current branch to `main`.
`git push -u origin main`	Upload your project to GitHub.
`git push --tags`	Upload your version tags.
`git pull`	Download and merge the latest remote changes.

Work Flow :

```
git status
git add .
git commit -m "Update Online Examination System"
git push
```

Week 01 : Project and HTTP Foundations :(Check the following exists before running the project) :

Goal: Understand the application structure and check that the required tools are available.

No.	Command	Simple meaning
1	`code .`	Open the current project in VS Code (if the `code` command is installed).
2	`dir`	See the frontend, backend, database and Keycloak folders.
3	`python --version`	Check your Python version.
4	`node --version`	Check your Node.js version.
5	`npm --version`	Check your npm version.
6	`git --version`	Check Git.
7	`docker --version`	Check Docker.
8	`docker compose version`	Check Docker Compose.
9	`curl.exe -i http://localhost:8000/`	Send an HTTP GET request to your backend root.
10	`curl.exe -i http://localhost:8000/docs`	Check whether the Swagger documentation URL responds.

Useful URLS to understand :(From our project) :

URL	Simple meaning
`http://localhost:3000`	React frontend.
`http://localhost:8000`	FastAPI backend.
`http://localhost:8000/docs`	Swagger API testing page.
`http://localhost:8080`	Keycloak login and administration.
`localhost:5432`	PostgreSQL database port.

Remember: `localhost` means your own computer. A port number identifies the application/service you want to reach.

Week 02 : Full commands with meaning : (APIs, data and architecture) :

Resource	What it stores
Users	Student and admin information.
Exams	Examination title, description and duration.
Questions	Questions belonging to an exam.
Submissions	Answers submitted by students.
Results	Marks and examination results.

Endpoints :

Operation	Example endpoint	Meaning
List exams	`GET /api/exams`	View exams.
Create exam	`POST /api/exams`	Add an exam.
View questions	`GET /api/exams/{exam_id}/questions`	View exam questions.
Submit exam	`POST /api/exams/{exam_id}/submit`	Submit answers.
View results	`GET /api/results/me`	View your results.
Admin action	`GET /api/admin`	Test admin-only access.

Concepts to Remember : (Week 2) :

Concept	Simple meaning
REST	A common way to design web APIs.
JSON	A format used to send data.
CRUD	Create, Read, Update and Delete.
Authentication	Check who you are.
Authorization	Check what you are allowed to do.
JWT	A signed token carrying identity and other claims.
RBAC	Give permissions based on roles.
ABAC	Give permissions based on attributes and context.
Serialization	Convert data into a format that can be sent.
Deserialization	Convert received data back into usable data.

For your project, an example RBAC rule is: **admin can create exams; student can take exams.**

Week 04 : (Build your FastAPI backend) : Full commands with meaning :

Goal: Create APIs, connect the database, validate data and test CRUD.

Prepare Python backend :

No.	Command	Simple meaning
1	`cd .\backend`	Enter the backend folder.
2	`dir`	See backend files.
3	`python --version`	Check Python.
4	`python -m venv .venv`	Create a separate Python environment for this project.
5	`.\.venv\Scripts\Activate.ps1`	Activate the environment in PowerShell.
6	`python -m pip install --upgrade pip`	Update the Python package installer.
7	`pip install -r requirements.txt`	Install backend dependencies.
8	`pip freeze`	Display installed Python packages.
9	`pip freeze > requirements.txt`	Save installed packages to the requirements file.
10	`dir .\app`	Check your application files.
11	`cd ..`	Return to the project root.

Run FastAPI locally:

No.	Command	Simple meaning
12	`cd .\backend`	Enter backend folder.
13	`uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`	Start your FastAPI application locally.
14	`Ctrl + C`	Stop the local server.

Test the backend using PowerShell (Run these while your backend is running) :

No.	Command	Simple meaning
15	`curl.exe -i http://localhost:8000/`	Test the root endpoint.
16	`curl.exe -i http://localhost:8000/health`	Test health if implemented.
17	`curl.exe -i http://localhost:8000/docs`	Check Swagger responds.
18	`curl.exe -i http://localhost:8000/api/exams`	Test the exam-list endpoint if implemented.

Understanding CRUD Testing : (Use SWAGGER or Postman) :

No.	HTTP method	Example endpoint	Simple meaning	Typical success
19	GET	`/api/exams`	Read all exams.	`200 OK`
20	GET	`/api/exams/{exam_id}`	Read one exam, if implemented.	`200 OK`
21	POST	`/api/exams`	Create an exam.	`201 Created`
22	PUT	`/api/exams/{exam_id}`	Replace an exam, if implemented.	`200 OK`
23	PATCH	`/api/exams/{exam_id}`	Change part of an exam, if implemented.	`200 OK`
24	DELETE	`/api/exams/{exam_id}`	Delete an exam, if implemented.	`204 No Content`

Useful status codes :

Code	Simple meaning
`400`	Request is not acceptable for the business rules.
`401`	Login/token is missing or invalid.
`403`	You are logged in but not allowed to do this.
`404`	Requested item does not exist.
`422`	Request data failed validation.
`500`	Something went wrong on the server.

Test PostgreSQL : (Your Compose service is named postgres, and the database is examdb) :

No.	Command	Simple meaning
25	`docker compose exec postgres pg_isready -U postgres -d examdb`	Check whether PostgreSQL is ready.
26	`docker compose exec postgres psql -U postgres -d examdb`	Open the PostgreSQL command line.

Inside PostgreSQL : Type :

```sql
\dt
```

Shows the tables.

```sql
SELECT current_database();
```

Shows the current database name.

```sql
SELECT * FROM exams;
```

Shows exam records, if the table exists.

```sql
\q
```

Week 05 : Docker and containerisation :

Goal: Package your backend and frontend so they can run inside containers.

Check docker :

No.	Command	Simple meaning
1	`docker --version`	Check Docker version.
2	`docker info`	Check whether Docker Engine is running.
3	`docker images`	Show downloaded and built images.
4	`docker ps`	Show running containers.
5	`docker ps -a`	Show all containers, including stopped ones.

Build your FastAPI backend image :

Run from your project root:

No.	Command	Simple meaning
6	`cd .\backend`	Enter backend folder.
7	`dir`	Check Dockerfile and requirements.
8	`docker build -t exam-backend:1.0 .`	Build the backend Docker image.
9	`docker images`	Check that the image was created.
10	`cd ..`	Return to project root.

Build your React frontend image :

No.	Command	Simple meaning
11	`cd .\frontend`	Enter frontend folder.
12	`dir`	Check frontend files.
13	`docker build -t exam-frontend:1.0 .`	Build your React Docker image.
14	`docker images`	Check the frontend image.
15	`cd ..`	Return to project root.

Run a separate backend container for practice :

Only do this if the Compose backend is not already using port 8000.

powershell

```
docker run -d --name exam-backend-practice -p 8000:8000 exam-backend:1.0
```

No.	Command	Simple meaning
16	`docker ps`	Check the container is running.
17	`docker logs exam-backend-practice`	View backend logs.
18	`docker logs -f exam-backend-practice`	Follow logs live.
19	`docker exec -it exam-backend-practice sh`	Open a shell inside the container.
20	`exit`	Leave the container shell.
21	`docker stop exam-backend-practice`	Stop the container.
22	`docker rm exam-backend-practice`	Remove the stopped container.

Docker volumes and networks :

```powershell
docker network create exam-practice-net
```

Creates a network for practice containers.

```powershell
docker volume create exam-practice-pgdata
```

Creates persistent database storage.

```
docker run -d --name exam-practice-db --network exam-practice-net -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=ExamDb123 -e POSTGRES_DB=examdb -v exam-practice-pgdata:/var/lib/postgresql/data postgres:16
```

Starts a separate PostgreSQL practice container with a named volume.

No.	Command	Simple meaning
23	`docker ps`	Check the practice database container.
24	`docker logs exam-practice-db`	Read its logs.
25	`docker exec -it exam-practice-db psql -U postgres -d examdb`	Open its database terminal.
26	`docker volume ls`	List Docker volumes.
27	`docker network ls`	List Docker networks.

Inside the practice database, test persistence.

</> SQL

```sql
CREATE TABLE test_data (id SERIAL PRIMARY KEY, message TEXT);
INSERT INTO test_data (message) VALUES ('My data is saved');
SELECT * FROM test_data;
\q
```

Now remove only the practice database container:

```powershell
docker rm -f exam-practice-db
```

Recreate it using the same volume.

```
docker run -d --name exam-practice-db --network exam-practice-net -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=ExamDb123 -e POSTGRES_DB=examdb -v exam-practice-pgdata:/var/lib/postgresql/data postgres:16
```

Check the saved row

```
docker exec -it exam-practice-db psql -U postgres -d examdb -c "SELECT * FROM test_data;"
```

Docker Hub commands :

Replace YOUR_DOCKERHUB_USERNAME with your real Docker Hub username.

No.	Command	Simple meaning
28	`docker login`	Sign in to Docker Hub.
29	`docker tag exam-backend:1.0 YOUR_DOCKERHUB_USERNAME/online-exam-backend:1.0`	Give your image its Docker Hub name.
30	`docker push YOUR_DOCKERHUB_USERNAME/online-exam-backend:1.0`	Upload the image to Docker Hub.

Week 6 : Docker Compose and complete project :

Goal: Run PostgreSQL, Keycloak, FastAPI and React together using one command.

Your Compose service names are:

```
postgres
keycloak
backend
frontend
```

Check your Compose project

Run from your project root:

No.	Command	Simple meaning
1	`cd "C:\Users\akash\OneDrive\Desktop\Project_Intern_Final_Wk_All\Exam\Exam"`	Go to your project root.
2	`dir`	Check project files.
3	`docker compose config --services`	List the services in your Compose file.
4	`docker compose config`	Check and display the combined Compose configuration.
5	`docker compose build`	Build your frontend and backend images.
6	`docker compose up -d`	Start all four services in the background.
7	`docker compose ps`	Check all service statuses.
8	`docker ps`	Check running containers.

View service logs :

No.	Command	Simple meaning
9	`docker compose logs postgres`	See PostgreSQL logs.
10	`docker compose logs keycloak`	See Keycloak logs.
11	`docker compose logs backend`	See FastAPI logs.
12	`docker compose logs frontend`	See React/Nginx logs.
13	`docker compose logs -f backend`	Follow backend logs live.
14	`docker compose logs --tail=100`	Show the last 100 log lines.

Press `Ctrl + C` to stop following logs.

Enter containers :

No.	Command	Simple meaning
15	`docker compose exec backend sh`	Open a shell inside FastAPI.
16	`exit`	Leave the backend shell.
17	`docker compose exec postgres psql -U postgres -d examdb`	Open PostgreSQL.
18	`\dt`	Show database tables.
19	`\q`	Exit PostgreSQL.

Test your four services :

No.	Command or action	Simple meaning
20	`curl.exe -i http://localhost:8000/`	Test FastAPI root.
21	`curl.exe -i http://localhost:8000/health`	Test health if implemented.
22	`curl.exe -i http://localhost:8000/docs`	Test Swagger.
23	Open `http://localhost:3000`	Open your React website.
24	Open `http://localhost:8080`	Open Keycloak.
25	`docker compose exec postgres pg_isready -U postgres -d examdb`	Check PostgreSQL readiness.

Test Keycloak and generate a token :

Your Compose file uses:

Realm: exam-system
Client: exam-frontend

- Open Keycloak:
- Keycloak Admin Console
- Check that your realm, client, users and roles exist.

```
$body = @{
   client_id  = "exam-frontend"
   grant_type = "password"
   username   = "admin"
   password   = "YOUR_ADMIN_USER_PASSWORD"
   scope      = "openid profile email"
}

$response = Invoke-RestMethod `
   -Method Post `
   -Uri "http://localhost:8080/realms/exam-system/protocol/openid-connect/token" `
   -Body $body `
   -ContentType "application/x-www-form-urlencoded"
```

```
$TOKEN = $response.access_token
```

Simple meaning: Keycloak checks the user credentials and returns an access token.

Test protected FastAPI endpoints

If /api/me exists in your Swagger page, run:

```powershell
curl.exe -i "http://localhost:8000/api/me" -H "Authorization: Bearer $TOKEN"
```

Simple meaning: Send the Keycloak access token to FastAPI to test authentication.

For admin and student permission tests, generate a separate token for each user and compare the results.

Test	Expected result
No token on a protected endpoint	`401 Unauthorized`
Invalid token	`401 Unauthorized`
Admin on admin-only endpoint	`200 OK`, if implemented and permitted
Student on admin-only endpoint	`403 Forbidden`, if role checks are implemented

Stop and restart the project

No.	Command	Simple meaning
26	`docker compose down`	Stop and remove project containers and network.
27	`docker compose up -d`	Start the project again.
28	`docker compose up -d --build`	Rebuild images and start the project.
29	`docker compose ps`	Check the service statuses.
30	`docker volume ls`	Check persistent volumes.
31	`docker network ls`	Check Docker networks.

Your project URLs


